

AWS Developer Exam: Domain 2 – Manage Sensitive Data in Application Code

Overview

Managing sensitive data in application code requires foundational development practices that protect data at every stage. This task statement covers:

- Data classification and its role in security
- Defense in depth using preventative and detective controls
- Discovering and cataloging sensitive data
- Securely handling credentials and secrets in application code
- AWS Well-Architected Framework recommendations for data protection

Why Data Classification Matters

Not all data is created equal. Properly classifying data is crucial to its security because it allows you to:

- Define appropriate **security zones** for different types of data
- Apply the right level of **network flow controls** and **access policy controls**
- Meet **regulatory and compliance obligations** (e.g., GDPR, HIPAA, PCI-DSS)
- Prevent mishandling of sensitive information

What to Understand About Your Data

Before implementing security controls, you must know:

- The **type and classification** of data your workload is processing
- The associated **business processes** and **data owners**
- Applicable **legal and compliance requirements**
- Where the data is stored** (S3, RDS, DynamoDB, etc.)
- The **resulting controls** needed to protect it
- Think about the data you are protecting, how it is stored, and who has access to it.

Defense in Depth

Defense in depth refers to **layering multiple security controls** together to provide redundancy in case a single control fails. There are two categories of security controls:

1. Preventative Controls

Designed to stop security incidents before they happen. The three main categories are:

IAM (Identity & Access Management)

Use MFA for authentication and authorization.

Apply the **principle of least privilege** throughout.

Initiate database connections and management operations from a **dedicated network zone** that is separate from application flows. This limits the damage if an application is compromised — management credentials and operations remain isolated.

Optionally, implement a **database firewall** as a policy enforcement point to ensure that only appropriate database commands are executed from a given security zone.

When a database requires credentials not integrated with IAM, use **Secrets Manager** with **KMS encryption**.

Infrastructure Security

Segment networks into zones appropriate for the sensitivity of the workloads running in them.

Separate management flows from application flows.

Data Protection

Apply both **encryption** and **tokenization** as layers of data protection.

2. Detective Controls

Designed to detect and alert on security events after they occur. An effective security posture requires **both preventative and detective controls** because the business and technology landscape are dynamic and evolving.

Examples of Detective Controls

Tool	Purpose
VPC Flow Logs	Continuous monitoring to identify unauthorized traffic and security anomalies
Amazon GuardDuty	Managed threat intelligence to detect malicious activity in the cloud
Amazon CloudWatch Events	Consume GuardDuty findings and drive automated responses
AWS Config	Detect configuration drift at the resource level
AWS Config can detect configuration drift in DynamoDB at the table level, and in RDS for DB instances, security groups, snapshots, subnet groups, and event subscriptions.	

Discovering and Classifying Sensitive Data

Step 1 — Identify and Discover Data with Amazon Macie

Amazon Macie is an AWS service that uses machine learning to automatically recognize sensitive data such as:

Personally Identifiable Information (PII)

Intellectual property

Financial data, credentials, and more

Macie should also be used for **continuous monitoring** of data security, usage, and access patterns.

Step 2 — Build a Data Catalog with AWS Glue

After identifying and classifying your data, establish a **data catalog** by inventorying:

Data types and how they are used

Compliance implications

Data ownership and retention requirements

Store this catalog information in the **AWS Glue Data Catalog**.

Step 3 — Label Data with SageMaker and Glue

Use **Amazon SageMaker** and **AWS Glue** together to add **data labeling**, ensuring data is classified and labeled appropriately at ingestion.

Step 4 — Continuous Monitoring

Continuously monitor data security, usage, and access patterns using **Amazon Macie** on an ongoing basis.

Building a Secure Data Pipeline (S3 Data Lake Example)

When building an automated data pipeline to prevent sensitive data from being ingested into a data lake, unstructured data (process reports, transcripts, emails, etc.) presents unique challenges for redaction and tokenization.

Recommended Pipeline Architecture with Macie

Ingestion — Bring data into the data lake from various sources.

Storage — Store data durably and securely in **S3 buckets** with appropriate access controls.

Processing — Transform data into a consumable state through validation, cleanup, normalization, transformation, and enrichment. This is where **Macie** adds a validation checkpoint to identify any sensitive data that has **not been appropriately redacted or tokenized** before consumption.

Consumption — Provide tools (analytics, BI, ML) to gain insights from the processed data in the data lake. Macie acts as an additional checkpoint in the processing layer to catch sensitive data that slipped through before it reaches consumption tools.

Securely Handling Sensitive Data in Application Code

Why Plaintext is Dangerous

Passing sensitive data (credentials, secrets, API keys) in plaintext can cause security issues because it becomes discoverable in the **AWS Management Console** or through AWS APIs such as

`DescribeTaskDefinition` or `DescribeTasks`.

Best Practice: Use Environment Variables with Secrets Manager or SSM Parameter Store

A security best practice — especially for containers — is to pass sensitive information as **environment variables** that reference values stored in Secrets Manager or SSM Parameter Store, rather than hardcoding them.

How to Inject Secrets by Service

AWS Service	How to Inject Secrets
Amazon ECS	Reference Secrets Manager or SSM Parameter Store in the task definition; expose as environment variables or in log config
AWS Batch	Reference secrets in the job definition
AWS CloudFormation	Retrieve secrets to use in other CloudFormation resources
AWS Lambda	Use the AWS Parameters and Secrets Lambda Extension to retrieve and cache Secrets Manager secrets

Additional Security Tips for Secrets Manager

Use an **interface VPC endpoint** to privately access Secrets Manager APIs without needing an internet gateway, NAT device, or VPN connection.

When configuring **Parameter Store with Secrets Manager**, the `secret-id` in Parameter Store requires a forward slash (`/`) before the name string (e.g., `/myapp/dbpassword`).

Secrets Manager vs. SSM Parameter Store — When to Choose Which

Consideration	SSM Parameter Store	Secrets Manager
Data always encrypted	Optional (plaintext or SecureString)	Always encrypted
Resource-level access policies	No	Yes
Automatic secret rotation	No	Yes
Best for	Configuration data, shared parameters	Sensitive credentials requiring encryption and rotation

Secrets Manager is often a better fit when the data must always be encrypted and resource-level access policies are needed.

Secure Credential Handling for People (Not Just Applications)

AWS requires different types of credentials depending on how you access AWS and what type of user you are.

AWS User Types and Credential Types

User Type	Credential Type
Account Root User	Username + password; MFA
IAM User	Username + password (console); Access keys (programmatic)
IAM Identity Center User	SSO credentials
Federated Identity	Temporary credentials via STS

Root User Best Practice

Delete the root user access keys immediately. You cannot change the permissions of the root user — it has full access to everything. Use it only for tasks that specifically require it, and protect it with MFA.

On-Premises Application Needing AWS Access

For AWS-hosted resources needing programmatic access, **use IAM roles** (best practice).

For **on-premises applications** needing AWS SDK access, **IAM roles cannot be used** — instead:

Create a new IAM user with programmatic access in the AWS console.

Generate access keys for that user.

Create a credentials file on the on-premises application server with those access keys.

Always follow least privilege — grant only the specific permissions needed for the task.

AWS Well-Architected Framework — Data Protection

Recommendations

Under the **Security Pillar**, AWS recommends four steps for data protection:

Step 1 — Classify Data

Categorize data into sensitivity levels.

Apply mechanisms such as **encryption**, **tokenization**, and **access control** based on classification.

Step 2 — Define Data Protection Controls

Controls to consider:

Tags — for resource identification and policy enforcement

Policies and AWS Organizations / SCPs — for organizational guardrails

KMS — for encryption key management

The goal is to **remove direct access** and **minimize manual processing** of sensitive data.

Step 3 — Define a Data Lifecycle Management Strategy

Based on sensitivity level and legal or organizational requirements, consider:

Retention — how long to keep data

Destruction — how and when to securely delete data

Access management — who can access data at each stage

Transformation — how data changes as it moves through your pipeline

Sharing — under what conditions data can be shared and with whom

Step 4 — Automate Identification and Classification

Use services to automate the detection and classification of sensitive data:

Amazon Macie — automatic sensitive data discovery

AWS Lambda — trigger automated responses

Amazon CloudWatch — monitoring and alerting

Amazon SNS — notifications and alerting pipelines

Key AWS Security Services to Know for the Exam

Beyond the services already covered, dive deeper into:

Service	Purpose
IAM	Authentication, authorization, least privilege
AWS Organizations	Multi-account management with SCPs
AWS Glue	Data cataloging and ETL
Amazon Neptune	Graph database (secure data relationships)
AWS CloudTrail	API call logging and audit trail
AWS Systems Manager	Parameter Store, Session Manager, patch management
Amazon GuardDuty	Managed threat detection
AWS Config	Configuration compliance and drift detection
AWS WAF	Web Application Firewall; policy enforcement at the edge
Amazon Macie	Sensitive data discovery and classification in S3

Key Exam Tips

Data classification is the first step in any security strategy — understand the type, sensitivity, owner, and compliance requirements of your data before applying controls.

Defense in depth = layering both preventative and detective controls. Neither alone is sufficient.

Amazon Macie is the go-to service for discovering and continuously monitoring PII and sensitive data in S3.

AWS Glue Data Catalog stores the inventory and metadata of your classified data assets.

Never pass secrets in plaintext — always reference Secrets Manager or SSM Parameter Store in ECS task definitions, Batch job definitions, Lambda extensions, etc.

Secrets Manager always encrypts data; SSM Parameter Store encryption is optional. Use Secrets Manager when you need both guaranteed encryption and automatic rotation.

Delete root user access keys — the root user cannot have its permissions scoped and should never be used for day-to-day operations.

For **on-premises applications** needing AWS SDK access, use IAM user access keys (not IAM roles, which require an AWS-hosted runtime).

AWS Config is the right tool for detecting configuration drift in RDS, DynamoDB, and other resources.

Follow the **four Well-Architected data protection steps**: classify → define controls → define lifecycle → automate. EOF